

1.Longest Increasing Continues Sublist

Aim: To find the length of the longest increasing continues sublist.

```
lst = [1, 2, 3, 1, 2, 3, 4]

max_len = 1
curr = 1

for i in range(1, len(lst)):
    if lst[i] > lst[i - 1]:
        curr += 1
        max_len = max(max_len, curr)
    else:
        curr = 1

print(max_len)

4
```

Result: Thus, the length of the longest increasing continues sublist was found and the program was executed successfully.

2.Check Whether One String Is a Rotation of Another

Aim: To check whether one string is a rotation of another string.

```
s1 = "abcd"
s2 = "cdab"

if len(s1) == len(s2) and s2 in (s1 + s1):
    print(True)
else:
    print(False)

True
```

Result: Thus, the given strings were checked for rotation and the program was executed successfully.

3.Elements Appearing More Than $n/3$ Times

Aim: To find the elements that appear more than $n/3$ times in a list.

```
lst = [1, 2, 3, 1, 1, 2, 1]
n = len(lst)
result = []

for x in lst:
    if x not in result and lst.count(x) > n // 3:
        result.append(x)

print(result)

[1]
```

Result: Thus, the required elements were found and the program was executed successfully.

#4.Longest Prefix That Is Also a Suffix

Aim: To find the longest prefix that is also a suffix of a string.

```
s = "abacaba"
n = len(s)
ans = ""

for i in range(1, n):
    if s[:i] == s[n-i:]:
        ans = s[:i]

print(ans)

aba
```

Result: Thus, the longest prefix that is also a suffix was found and the program was executed successfully.

5.Maximum Product of Any Three Numbers

Aim: To find the maximum product of any three numbers in a list.

```
lst = [-10, -3, 5, 6, -2]

lst.sort()

prod1 = lst[-1] * lst[-2] * lst[-3]
prod2 = lst[0] * lst[1] * lst[-1]
```

```
print(max(prod1, prod2))  
180
```

Result: Thus, the maximum product of three numbers was found and the program was executed successfully.

6. Word with the Most Repeating-Character Occurrences

Aim: To find the word with the highest number of repeating characters.

```
sentence = "apple banana committee"  
words = sentence.split()  
  
max_word = ""  
max_count = 0  
  
for w in words:  
    count = 0  
    for ch in w:  
        if w.count(ch) > 1:  
            count += 1  
  
    if count > max_count:  
        max_count = count  
        max_word = w  
  
print(max_word)  
committee
```

Result: Thus, the required word was found and the program was executed successfully.

7. Minimum Swaps Required to Sort the List

Aim: To find the minimum number of swaps required to sort a list.

```
lst = [4, 3, 2, 1]  
swaps = 0  
arr = lst[:]  
  
for i in range(len(arr)):
```

```

while arr[i] != i + 1:
    j = arr[i] - 1
    arr[i], arr[j] = arr[j], arr[i]
    swaps += 1

print(swaps)
2

```

Result: Thus, the minimum number of swaps was found and the program was executed successfully.

8. Maximum Number After Removing One Digit

Aim: To find the maximum number after removing one digit.

```

s = "52743"
best = "0"

for i in range(len(s)):
    temp = s[:i] + s[i+1:]
    if temp > best:
        best = temp

print(best)
5743

```

Result: Thus, the maximum number was obtained by removing one digit and the program was executed successfully.

9. Find All Palindromic Substrings

Aim: To find all palindromic substrings in a given string.

```

s = "ababa"
palindrome = []

for i in range(len(s)):
    for j in range(i + 1, len(s) + 1):
        sub = s[i:j]
        if sub == sub[::-1] and len(sub) > 1:
            palindrome.append(sub)

print(palindrome)

```

```
['aba', 'ababa', 'bab', 'aba']
```

Result: Thus, all palindromic substrings were found and the program was executed successfully.

10. Maximum Difference with a Larger Right Index

Aim: To find the maximum difference with a larger right index in a list.

```
lst = [2, 3, 10, 6, 4, 8, 1]

min_val = lst[0]
max_diff = lst[1] - lst[0]

for i in range(1, len(lst)):
    max_diff = max(max_diff, lst[i] - min_val)
    min_val = min(min_val, lst[i])

print(max_diff)

8
```

Result: Thus, the maximum difference was found and the program was executed successfully.